

A Real-time Hand Gesture Recognition System Implemented on the MARRtino Mobile Robot in a Simulated Hospital

Christopher Kreienkamp¹

¹Department of Computer, Control and Management Engineering, Sapienza University of Rome, Via Ariosto 25, Roma, 00185, Italy

Abstract

A nurse must complete a mix of critical and non-critical tasks. This paper presents a simulation of the MARRtino mobile robot that can safely path plan to a list of goal locations in a hospital to complete routine non-critical tasks. Further, a hybrid neural network model is developed that uses the monocular video input from MARRtino to extract hand features with MediaPipe hands and then classify hand gestures with a two-layer GRU. The incorporation of the gesture recognizer enables medical professionals to interact with MARRtino and edit goals in real-time. This demonstrates just one of a large variety of ways that alternative forms of human-robot interaction can assist nurses in critical care environments.

Keywords

Hand Gesture Recognition, Machine Learning, Gate-Recurrent Neural Networks, MediaPipe Hands

1. Highlights

- modifies the MARRtino robot and AWS hospital world models to be compatible with ROS2 and Gazebo Sim
- simulates MARRtino path-planning in the AWS hospital world, assessing the ability for the workstation to handle the processing
- proposes a novel way to assign tasks to a medical robot
- develops a neural network model to recognize hand gestures
- demonstrates how the model, trained on a specific length of image frames, can be extended for use with a continuous video feed

2. Introduction

Healthcare facilities, such as hospitals, rehabilitation centers, and nursing homes, are intensely challenging work environments for nurses. Nursing requires constant and intense focus for the entirety of a 12 hour shift, managing tasks in the midst of unpredictable interruptions. A small mistake in a routine activity such as administering medication could have fatal consequences.

The hospital environment therefore is a good target for incorporating automation to complete repetitive and non-critical tasks that nurses currently perform. Removing such tasks from a nurse's workload would free time and mental resources for nurses to dedicate to more patient-facing critical tasks. Ventura et al. studied nursing workflows to identify the most time-consuming and stressful parts of the job [1]. They identified that in an average 12 hour shift, nurses perform the following tasks (hours:minutes):

- 4:08 - communication (e.g. with patients, doctors)
- 3:06 - documentation (e.g. patient status, plan)
- 3:02 - non-delegable tasks (e.g. giving medication)
- 0:26 - delegable tasks (e.g. feeding, checking vitals)
- 0:24 - non-nursing tasks (e.g. delivering food)

Documentation, delegable tasks, and non-nursing tasks would be good contenders for robotic automation because

they are all non-critical activities that do not require the human social element. Eliminating these tasks could free nearly 4 hours of time in every nurse's shift.

Transportation tasks, such as retrieving food, water, medicine, linens, and lab samples, are a good first choice for automation. These tasks are highly predictable and do not interface with a patient in a complex manner. A secondary choice is room sanitization, especially if the hospital rooms are uniform in their size and setup. The robot therefore must be mobile, able to navigate in the dynamic hospital without collision. It must also be low-cost, to enable a hospital to purchase a swarm, so that the robots can prioritize and distribute tasks efficiently.

Nurses overwhelmingly voted that documentation, or charting, was the most burdensome task [1]. With it being the second most time-consuming category as well, it also makes sense to develop methods to automate and quicken documentation. Nurses document at the nurses station 53% of the time, in the hallway 26% of the time, in patient rooms 14% of the time, and 7% in other locations [1]. A more seamless and uninterrupted workflow would involve documenting as much as possible immediately within a patient's room, lowering the possibility of forgetting a crucial piece of information or charting for the wrong patient.

In studying the usability of documentation systems, one should explore alternative methods for documenting. Before computers, and even now when computer systems break down, nurses had to write everything by hand. Computers helped to organize and extend the thorough nature of documentation, with nurses typing in their notes with a keyboard. This, however, is not the fastest and safest way to document. Thumann et al. analyzed documentation in dermatology, and discovered that speech recognition technology could produce a page of documentation in 15.8 minutes, while traditional methods took 24.1 minutes per page [2]. Even when speech recognition does not speed up the documentation process, there are less workflow interruptions when it is used, as it is frequently done in the room with a patient as soon as the nurse has information to report. Patients even reported a high rate of satisfaction and a feeling of respect when the dictation was done in their presence [3]. With the increase in output, speech recognition undoubtedly has the potential to create more errors in the documents, creating potentially confusing errors in up to 22% of radiology reports, one study concluded [4]. Even with these extra errors, the output from the speech recognition software could be

Elective in Artificial Intelligence, Fall 2024

✉ kreienkamp.1979531@studenti.uniroma1.it (C. Kreienkamp)



© 2025 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

sent to a human transcriptionist to edit the errors. When this system was put in place, radiologists using speech recognition that was sent to an editor were able to produce 41% more reports than their counterparts.

Because of its obvious advantages, speech recognition for documentation has been extensively studied, and hospitals are slowly adopting the technology to various degrees. A less-well studied form of alternative interaction is with hand gestures. Hand-gestures could additionally improve the documentation process because a nurse could navigate through a document on a computer with the hand gestures, and then dictate to fill in specific text boxes. This is desirable in a hospital environment because hand gestures, like speech recognition, are a contactless form of communication. Many medical professionals use a shared computer in the hallways and in the patient rooms. By constantly administering to several different patients, they inevitably deposit germs onto the keyboard and mouse. This is a health concern, and the existence of bacteria on these surfaces in hospitals heavily documented [5] [6] [7].

The use of hand gestures in hospitals does not need to be limited just to documentation. The hand gestures could be used to navigate computer screens and select actions, make lab orders, and page doctors. With such a wide possible modes of usage, this paper explores hand gestures specifically, demonstrating how a hybrid machine learning model can detect hand gestures from a monocular video and translate these into computer commands.

Returning to the earlier-mentioned practical goal of developing a transportation robot in a hospital, this paper demonstrates the incorporation of a hand gesture model into a mobile robot. The mobile robot is simulated in a hospital environment, starting at the nurse's station, and given a list of tasks to complete. In this case, a task is considered complete when the robot reaches a specific goal location in the hospital, though a gesture or audio cue could be implemented for the robot to receive confirmation of successfully completing a task. In this implementation, the list of tasks given to the robot can be edited using hand gestures. This might not be the most ideal application in the real-world, but it offers a convenient way to demonstrate the successful integration of hand gesture recognition technology into hospital robots.

3. Related Works

This paper explores two separate topics: robots used in healthcare and gesture recognition technologies.

Robot use within healthcare is not a novel idea, as Kyrarini et al. even classify the extensive number of healthcare robots into the categories of care, hospital, assistive, rehabilitation, and walking assisting [8]. The capabilities of these robots are very diverse, from general transportation to direct patient care.

When it comes to transportation robots, which Kyrarini would classify as hospital robots, the most basic and popular configuration takes the form of the TUG mobile robot from Aethon [9]. The robot is battery-powered, can safely avoid obstacles, and can even communicate to open doors and elevators that have a special module installed. As long as it has been initialized with an accurate map, it can handle the dynamic hospital environment by autonomously navigating around people and furniture, communicating its presence through audio cues. If needed, a human can take

over control of the robot through a Wi-Fi connection. To fulfill its transportation purposes, it comes with a variety of load-carrying modules, designed to be loaded and unloaded by a human. The pharmacy carts transport patient ordered prescription medications and lab samples, linen service carts transport clean and dirty laundry, and food service carts transport meals to patients and take dirty dishware back to the kitchen. At 19 cm high and 51 cm wide, the TUG robot can fit in tight spaces between hospital beds and other equipment.

The Moxi robot developed by Diligent Robotics, which is also classified as a hospital robot, is more advanced and has significantly greater functionality than the TUG robot due to its robotic arm [10]. With its own arm, the Moxi robot can transport the same things that the TUG robot can, and Moxi does not need human intervention to load and unload the items. Moxi can stock itself by accessing storage closets and stockrooms autonomously, further allowing medical staff to complete more critical and patient-facing tasks. Like the TUG robot, Moxi does not interact with patients. ABB aims to extend the benefits of adding robotic arms, developing the YuMI robot that can work in conjunction with hospital workers with its two robotic arms [11].

Once a safe mobile robot platform is created, transportation is easily equipped, as is demonstrated with the TUG robot and the added complexity of the the Moxi and YuMi robots. Another frequent and necessary task in a hospital is sanitization. Fraunhofer Institute for Manufacturing Engineering and Automation developed the prototype mobile robot DeKonBot, equipped with a robotic arm like Moxi [12]. Rather than for transportation, DeKonBot is designed to detect and wipe down frequently-touched surfaces like door handles with disinfectant. As is demonstrated here, a mobile robot, especially one with an arm, can be extended to fulfill a variety of specialized tasks as long as the initial platform can safely navigate autonomously.

The most popular method for a mobile robot navigation is to use a path planner to optimize the best path on an internal map of the environment in real time. This is not universally used, however, as Yildirim et al. conceptualized a medication-dispensing robot that uses an ANN-trained controller to track a permanent magnetic strip on the floor of the hospital [13]. Safe navigation through a crowded and dynamic hospital has complex demands, so some hospitals might prefer this more predictable approach. The TUG robot and other compact robots are ideal for their ability to maneuver in tight spaces. Users of the mobile robot HoLLiE reported a significant problem with its large dimensions, making it at times impossible to maneuver through pathways blocked by beds or other equipment [14].

The second topic explored in this paper, gesture recognition, also has been studied previously, offering the luxury of extensive datasets. A gesture recognition system can include more than just hand recognition. Quadriplegics could control the Jaco 2 robotic arm from Kinova [15] to bring a drinking glass to their lips with a head position estimator [16]. In this study, however, only hand dynamic hand gestures are recognized. Broadly, hand gesture recognition can be categorized as either identifying static hand poses or dynamic hand movements. A few more key factors further taxonomize the problem [17]:

- sensor input: RGB, RGB-D, or video, monocular or multi-view cameras
- task: classification of the gesture or estimation of

the spatial structure of the hand

- capture: representation of the hand as a skeleton or detection of the box/region where the hand exists in an image
- method: single neural network, traditional machine learning approach, hybrid use of several techniques

The sensor input and task are highly dependent on the given problem, but the capture and method are generally choices available to the engineer solving the problem. The most popular approach to solving the problem is a hybrid method where the feature extraction step, whether it be identifying the hand skeleton or region of interest, is separated from the classification step. Hybrid approaches typically increase accuracy and reduce cost [17].

MediaPipe hands is frequently used to extract the hand features from an image. MediaPipe hands is a trained model that detects 21 distinct landmarks from the palm and fingers in an RGB image [18]. Meng et al. for example, processed individual images with MediaPipe hands and extracted the region of interest containing the hand [19]. After extracting the region of interest, they cropped the original image and passed it to the Inception-v3 2D feature extractor. An LSTM was then used as a temporal classifier. Kumar et al. took a similar approach, using MediaPipe hands and then feeding the output to a CNN to for ASL gesture classification [20]. A GRU instead of an LSTM has also been used previously in conjunction with MediaPipe hands for detecting Indian sign language [21].

MediaPipe is popular because it is accurate and lightweight, but it is not required to solve the gesture recognition problem. Kopuklu et al. used a simple CNN to extract the hand from an image and a deep CNN to classify the gesture [22], while Singha et al. used skin filtering and palm-cropping to identify the location of the hand [23]. Models aimed at recognizing dynamic hand gestures generally achieve accuracies between 85% and 90% [24] [25] [26].

4. Proposed Method

To achieve the goal of a mobile hospital robot with gesture recognition capabilities, several distinct steps are accomplished. First, an accurate mobile robot model is simulated in a hospital, and then safely navigates through the rooms and hallways. Next the nurse sends tasks to the robot and modifies these tasks in real-time using a keyboard. Finally, the keyboard is replaced by a gesture recognizer to serve as the main form of interaction with the robot.

4.1. Simulating a robot in a hospital

The MARRtino robot was chosen for being mobile and cost-effective [27]. It is based on ROS and has been extended to many shapes, from a small box-like red robot that is low to the ground to a taller human-like white robot that includes an interactive screen and a screen to simulate a cartoon face. The taller robot, termed MARRtina, has been used in classroom environments, interacting with children in a safe and fun manner. Our implementation displayed in Fig. 1 is a mixture, containing parts of both the original MARRtino and the advanced MARRtina. It is taller like MARRtina, but it does not include the face or body-like structure. This implementation includes a camera, lidar sensor, and computer screen (not pictured).

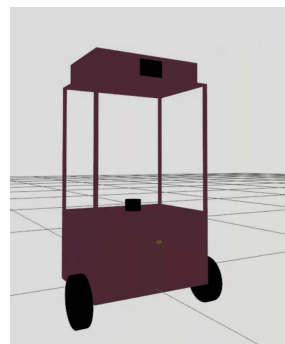


Figure 1: MARRtino robot: an open-source, open hardware differential-drive robot developed at the Sapienza University of Rome, Italy (<https://www.marrtino.org/>).



Figure 2: The AWS hospital world is an Amazon Web Services ROS package with models to simulate a hospital in Gazebo (<https://github.com/aws-robotics/aws-robomaker-hospital-world>).

For the simulation environment, the AWS hospital world in Fig. 2 was chosen because it is a complete and accurate model of a hospital floor with a nurse's station, patient rooms, closets, hallways, and furniture [28]. The AWS hospital world contains 12 patient rooms with hospital beds, 4 meeting rooms, 2 storage closets, a nurse's station, a general patient/family waiting area, and access to 2 stairwells. Using accurate simulation environments is crucial to safely testing a robot for all possible situations in the real world. Ideally, all failures occur in the simulation environment so changes can be made to avoid this in the real world, where it is costly and could destroy robot hardware or even worse, injure people.

Before proceeding with any simulation, the MARRtino robot and AWS world files, both designed for use with ROS1, needed to be updated to be compatible with ROS2 and Gazebo Sim. Robot Operating Systems (ROS) is one of the most popular frameworks for managing robotic projects, with specialized packages for simulation and control. Since the time when the MARRtino robot and AWS world were released, ROS1 has been modernized to ROS2 to include essential features and algorithms [29]. Though the syntactical and conceptual differences between ROS1 and ROS2 do not overhaul the entire original models for MARRtino and the hospital world, they are enough to break the models in their raw form.

After editing for ROS2 compatibility, MARRtino could successfully load into the world. Unfortunately, the computational power of the main workstation used for this study (Lenovo ThinkPad T490 with Intel® Core™ i5-8265U × 8 processor, 16G RAM) was not adequate to support the full

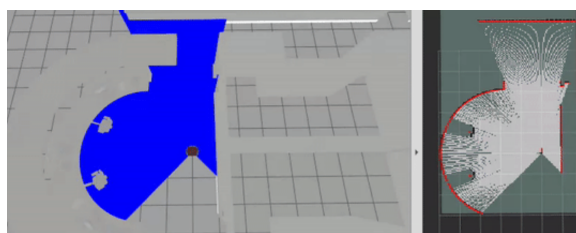


Figure 3: MARRtino using the ROS2 Nav2 stack to create a map of the hospital world using lidar sensor data.

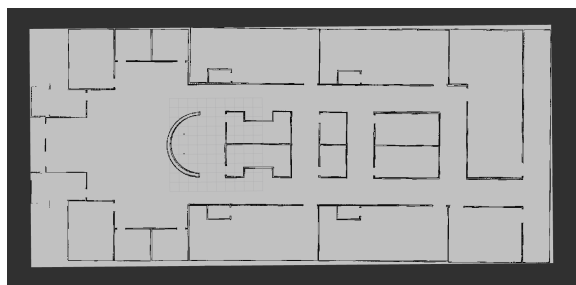


Figure 4: Completed map of the AWS hospital world. This is used by MARRtino to localize and path plan.

simulation with all of the hospital beds, chairs, and other furniture. All accessories and equipment were therefore removed, leaving just the structure of the hospital without any people or furniture.

4.2. Safely path planning to a goal position in the hospital

To make MARRtino extensible to many different hospitals, it was decided to navigate using the most popular approach, that is, to optimize a path on an internal map of the world. The method to follow a magnetic strip on the floor as demonstrated in [13] would be easier computationally for the robot, but each hospital would need to install this strip and could not immediately use MARRtino. To collect an accurate internal map of the world, the user controlled MARRtino with arrow keys using the ROS2 package `teleop_twist_keyboard` to visit every part of the hospital. While exploring, MARRtino collected lidar sensor data to simultaneously localize and map (SLAM) using the ROS2 Nav2 stack, as seen in Fig. 3 [30]. After mapping the entire world, the completed map in Fig. 4 was saved to use for future localization and path planning. Because of the large size of the world, it took around 2 hours to collect the required amount of sensor data.

With an initial pose estimate of its location in the map, MARRtino could localize itself using Nav2's default Adaptive Monte Carlo Localization (AMCL) 2D pose estimation algorithm. The user then specified a random point on the map as a goal pose, and the Nav2 stack used the real-time lidar data and localized pose to create a safe local and global path to the goal using the default Dijkstra's algorithm. Note that the camera data was not used for localization. A differential-drive controller tracked this path by actuating the wheel joints. On the proposed MARRtino robot, the two front wheels are controllable, while the back wheel is a passive caster wheel for support.

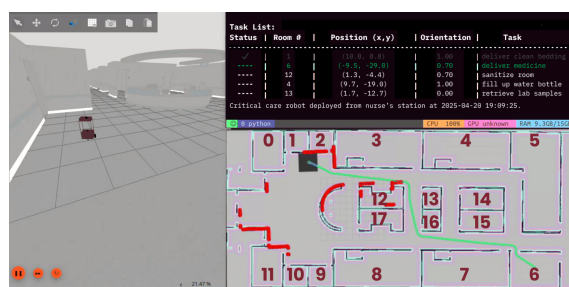


Figure 5: MARRtino completing the list of tasks by path-planning to the correct numbered room. In this case, the first task has been completed.

4.3. Send a list of tasks to the robot

After MARRtino could reach one goal location, several goal locations were given consecutively. These goal locations were formatted as tasks, meaning that each goal location was in a specific room where MARRtino needed to complete something. The possible transportation tasks involve retrieving or delivering food, water, medication, lab samples, linens, trash, or paper products. The only non-transportation task is room sanitization.

MARRtino was initialized to start at the nurse's station located at the center of the map. In a real hospital, the swarm of robots could be waiting on call in this area, sitting on charging pads. When ready, the nurse could give a list of tasks, each to be completed in one of the 18 rooms. Fig. 5 displays the simulation, in which random tasks were assigned in random rooms.

For the purposes of this study, a task was considered completed once MARRtino reached the goal location in a room, though this could be easily modified to require approval in the form of a gesture from a patient or medical professional in the room. The simulation also strayed from reality in that the tasks and goal locations were sent in a random order without consideration of the unique task. If a robot needed to deliver fresh linens to a room, for example, the robot would first need to travel to the storage closet, collect the clean linens, and then travel to the room. MARRtino could not travel directly to the room from the nurse's station in this case.

After the list was sent, the list could be edited using one of the four arrow keys on a keyboard: **up**, **down**, **left**, and **right**. These four inputs were intentionally chosen to prepare for the next section, and it will be explained why in that section. To make use of the small number of possible inputs, a system was developed that expanded the possible input space by the use of sequences. Rather than communicating with the robot with a singular **[up]** or **[down]** arrow, a series of arrow keys needed to be sent in order to command the robot to do something.

The robot could be stopped with a sequence **[up, down, up]** at which point it would wait for further input from a healthcare professional. If the robot is not stopped, its list of goals cannot be edited. **[up]** and **[down]** could navigate through the list of tasks, **[right, right]** could prioritize a task to make it first on the list, and **[left, left]** could eliminate a task that was no longer needed. After making edits, the staff member could send the robot to continue its mission with the updated list using the sequence **[up, down, up, down]**.

The use of sequences also makes the system more robust.

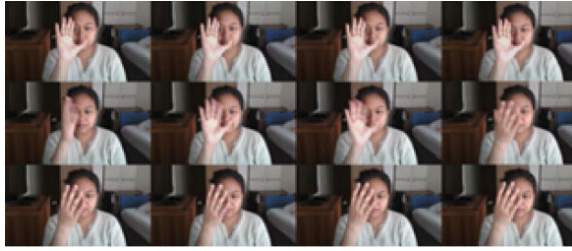


Figure 6: Sample video frames from the Jester Dataset V1 for Hand Gesture Recognition.

For example, if the robot is moving and should continue moving, it is much less likely to incorrectly interpret the sequence **[up, down, up]** than it is to incorrectly interpret a single gesture.

4.4. Edit the list of tasks with a gesture recognizer

The setup of the task editing process with arrow keys is well-suited to be replaced by a hand gesture recognizer that can distinguish four unique dynamic hand gestures: up, down, left, and right. The basic problem that we seek to solve with a hand gesture recognizer is to classify what hand gesture was made given video frames of a person making hand motions. The performance measure to judge the effectiveness of the hand gesture recognizer is accuracy. First raw hand gesture video data was prepared, then it was used by several neural networks to train, then the best-performing network was incorporated into the hospital robot in simulation.

4.4.1. Data preparation

The Jester Dataset V1 for Hand Gesture Recognition was used as the raw input data, composed of 50420 train and 7047 validation samples across 27 classes [31]. Each sample is a folder of 37 consecutive jpg frames from a video, with some of those frames shown in Fig. 6. A CSV file labels the hand gesture performed in the video.

To prepare the data, each jpg image was reshaped to $100 \times 176 \times 3$. Each image was then converted to a flattened numpy array of 21 hand keypoints coordinates (x, y, z) using MediaPipe hands, giving 63 values per frame. Keypoints were interpolated for frames where no hand was detected. MediaPipe hands automatically normalizes the position values between [0, 1].

Similar gestures were then grouped (e.g. Swiping Right, Sliding Two Fingers Right) and the rest were removed from the dataset to leave four desired gesture classes: right, left, down, up. To augment the training set, the frame sequence for every video was reversed such that a video classified as right, when reversed, could be classified as left. Finally, the categorical targets were one-hot encoded to transform the label into four binary columns. A complete visual of the data preparation process is displayed in Fig. 7.

The process of converting every frame to a flattened numpy array of 21 hand keypoints coordinates using MediaPipe took around 6 hours on our standard workstation. Time could have been saved if only the videos from relevant classes had been converted, as only 8 classes of the 27 were used for training. At the time when we extracted the hand keypoints, we had not yet made a final decision on the desired classes, and we did not want to limit our choices later

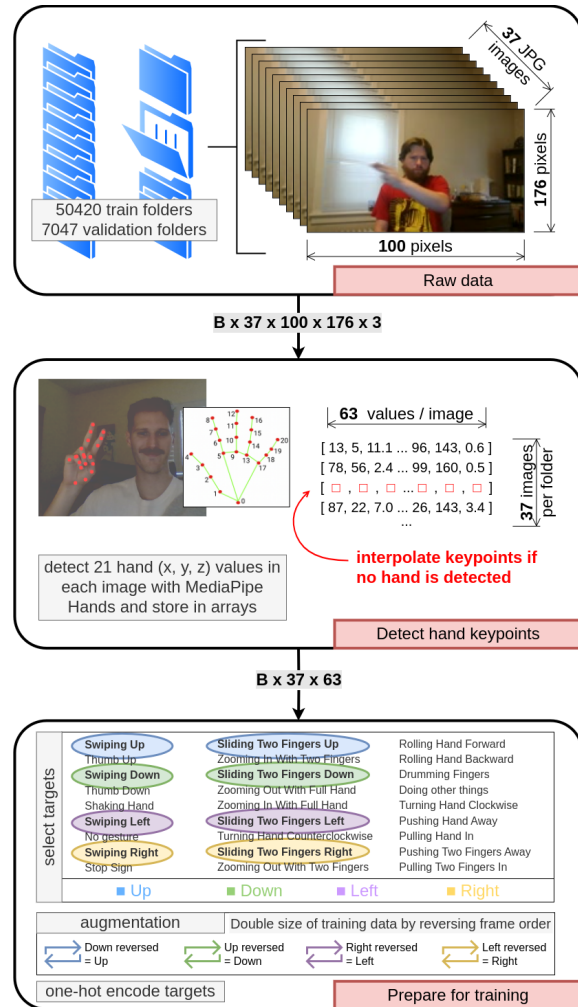


Figure 7: Data preparation pipeline. The raw data is originally in the form of folders each containing separate videos. These folders are converted into arrays containing the locations of hand keypoints.

on in the study. It would, however, save time and processing if the desired classes were selected before any frames were converted to numpy arrays of hand keypoints.

4.4.2. Model training

After preparing the data, a several models were investigated for their hand gesture recognition performance based on the input of the temporally located hand keypoints. One-layer and two-layer PyTorch implementations of the following neural networks were trained:

- Recurrent neural network (RNN)
- Long short-term memory (LSTM)
- Gated recurrent unit (GRU)
- Linear
- Convolution neural network (CNN)

Additionally, the scikit-learn algorithms [32]:

- Logistic regression
- Support vector machines (SVM)
- Decision trees

Algorithm	Train	Validation
Baseline - ZeroR	25.20%	25.21%
RNN (63)	58.96%	59.23%
RNN (128x128)	52.83%	54.94%
LSTM (63)	85.84%	82.64%
LSTM (128x128)	86.29%	83.88%
GRU (63)	86.23%	83.88%
GRU (128x128)	86.98%	84.86%
Linear (30)	82.50%	80.35%
Linear (2331)	84.87%	79.74%
Conv1d (30)	70.73%	66.82%
Conv1d (64x128)	78.01%	72.87%
Logistic Regression	–	78.00%
Support Vector Machine	–	67.60%
Decision Tree Classifier	–	64.30%

Table 1

Hand gesture recognition accuracies for a variety of one- and two-layer neural networks and scikit-learn machine learning algorithms.

The models were trained on an NVIDIA T4 GPU, accessed using Google Colab. Table 1 displays the accuracies of the models after training. As expected, the LSTM and GRU networks, which are ideal for sequential data, showed the highest accuracies. The loss, although not presented here, displayed similar trends to the accuracy data. Because of the high performance from the LSTM and GRU models, these two network topologies were explored more in-depth to discover better task performance.

Further tuning was done, and it was found that the two-layer GRU with hidden-layer sizes of 128 demonstrated the best performance. The model used cross entropy loss, an Adam optimizer, learning rate scheduler, and batch size of 128 while training.

4.4.3. Model deployment

The final problem to solve before incorporation into the robot was how to handle a video input. The neural network was trained for an input sequence of 37 frames, but the video is a continuous sequence.

This was solved by continuously detecting the MediaPipe hands keypoints, and only sending the keypoints to the model for classification if they met a very specific criteria. The criteria is as follows. At every frame, after the keypoints were detected, the keypoints were stored in a queue of length 37. The keypoints from 37 frames before were destroyed, while the keypoints from the most recent frame were added to the queue. The center of the hand for the first and last frames in this queue was found by averaging the keypoint locations for these frames. If the center of the hand was near the center of view (within a circle of some radius) at the oldest frame of the queue and far from the center of view (outside a circle of some radius) at the most recent frame of the queue, the queue was sent to the model for classification to output a gesture.

The idea behind this is that a hand must make a substantial and continuous movement that originates or passes through the center of the camera's field of view in order to be a gesture. It must be continuous because if a hand was not tracked in a frame, the entire queue was deleted and reset. This guaranteed that a hand had been tracked for 37 consecutive frames. The keypoints queue was chosen to have a length of 37 frames in order to match the input to



Figure 8: Loss and accuracy over 16 epochs for the training and validation of the model.

the model during training.

5. Results

A validation accuracy of 85.7% was achieved after training the two-layer GRU network for 16 epoch. The loss and accuracy for the training and validation datasets are displayed in Fig. 8. These plots show expected behavior. The validation accuracy follows the trend of the training accuracy, and it does not deviate significantly, meaning that the model is not overfitting. After training, the model was successfully incorporated into the simulation, as described in the methods section. The user was able to successfully edit the robot task list using gestures, and the use of sequences prevented incorrectly interpreted gestures from causing the robot to complete an unwanted action.

The hospital simulation did not include any accessories or equipment due to the inability of our under-powered workstation to process all of the extra simulation. When editing the robot task list using gestures, the simulation was slower than real-time, but we expect that this could be alleviated by using a more modern workstation better suited for the task.

6. Conclusions

This paper presented the incorporation of a hand gesture recognition system into a mobile robot that could safely path plan to a list of goal locations. The MARRtino robot and AWS hospital world files were updated to be compatible with ROS2 Jazzy and Gazebo Sim so that MARRtino could be simulated in the hospital. A hybrid approach was taken for the hand gesture recognition model, separating the hand feature extraction step from the temporal reasoning of the hand video sequence. MediaPipe hands extracted the hand features, and a variety of popular neural network architectures and machine learning algorithms were explored for the temporal reasoning part. Of all the classifiers tested, a two-layer GRU network demonstrated the highest success with an accuracy of 85.7%.

The creation of gesture sequences to trigger events, rather than singular motions, made the hand gesture system more robust, as two or more incorrectly-classified gestures would be required to trigger an unwanted event. The gesture recognition system was successfully incorporated into the mobile robot so that a medical professional could give commands to a robot in a contactless manner. In our case, we demonstrated how these commands could be used to edit a list of tasks that the autonomous robot must accomplish to relieve

nurses of non-critical duties like transporting drinking water to patients. Our implementation demonstrates just one of a large variety of ways to configure a mobile robot with a hand gesture recognition system to help nurses, minimize the chances for preventable mistakes, and avoid unnecessary spread of germs through contact in a hospital setting.

References

- [1] H. B. Ventura, N. Alexander, An observational study on nursing workflows: Task distribution and burden analysis (2024).
- [2] P. Thumann, S. Topf, A. Feser, C. Erfurt, G. Schuler, V. Mahler, Digital speech recognition in dermatology: a pilot study with regard to medical and economic aspects, *Der Hautarzt* 59 (2008) 131–134.
- [3] T. Waterston, Dictating clinic letters in front of the patient. further research should be qualitative, *BMJ: British Medical Journal* 314 (1997) 1416.
- [4] L. E. Quint, D. J. Quint, J. D. Myles, Frequency and spectrum of errors in final radiology reports generated with automatic speech recognition technology, *Journal of the American College of Radiology* 5 (2008) 1196–1199.
- [5] S. Bures, J. T. Fishbain, C. F. Uyehara, J. M. Parker, B. W. Berg, Computer keyboards and faucet handles as reservoirs of nosocomial pathogens in the intensive care unit, *American journal of infection control* 28 (2000) 465–471.
- [6] P.-L. Lu, L. Siu, T.-C. Chen, L. Ma, W.-G. Chiang, Y.-H. Chen, S.-F. Lin, T.-P. Chen, Methicillin-resistant staphylococcus aureus and acinetobacter baumannii on computer interface surfaces of hospital wards and association with clinical isolates, *BMC Infectious Diseases* 9 (2009) 1–7.
- [7] N. Ide, B. K. Frogner, C. M. LeRouge, P. Vigil, M. Thompson, What’s on your keyboard? a systematic review of the contamination of peripheral computer devices in healthcare settings, *BMJ open* 9 (2019) e026437.
- [8] M. Kyrarini, F. Lygerakis, A. Rajavenkatanarayanan, C. Sevastopoulos, H. R. Nambiappan, K. K. Chaitanya, A. R. Babu, J. Mathew, F. Makedon, A survey of robots in healthcare, *Technologies* 9 (2021) 8.
- [9] R. Bloss, Mobile hospital robots cure numerous logistic needs, *Industrial Robot: An International Journal* 38 (2011) 567–571.
- [10] Diligent Robotics, Moxi Helps Hospitals and Clinical Staff, <https://diligentrobots.com/moxi>, 2020. Accessed: 2025-05-30.
- [11] ABB Ltd., Abb demonstrates concept of mobile laboratory robot for hospital of the future, <https://new.abb.com/news/detail/37279/hospital-of-the-future>, 2020. Accessed: 2025-05-30.
- [12] Fraunhofer IPA, DeKonBot—Mobile Robot for Disinfecting Potentially Contaminated Surfaces, https://www.youtube.com/watch?v=IbJ6P9lfXDM&feature=emb_title&ab_channel=FraunhoferIPA, 2020. Accessed: 2025-05-30.
- [13] Ş. Yıldırım, S. Savaş, Design of a mobile robot to work in hospitals and trajectory planning using proposed neural networks predictors, in: *International Conference on Reliable Systems Engineering*, Springer, 2021, pp. 32–45.
- [14] L. Steffen, M. Schulze, C. Eichmann, R. Koch, A. Hermann, R. Frietsch Mussulin, F. Graaf, R. Wilbrandt, M. Große Besselmann, A. Roennau, et al., Hollie c—a multifunctional bimanual mobile robot supporting versatile care applications, in: *International Conference on Intelligent Autonomous Systems*, Springer, 2023, pp. 127–140.
- [15] Kinova Robotics, Kinova jaco assistive robotic arm, <https://www.kinovarobotics.com/en/products/assistive-technologies/kinova-jaco-assistive-robotic-arm>, 2020. Accessed: 2025-05-30.
- [16] F. F. Goldau, T. K. Shastha, M. Kyrarini, A. Gräser, Autonomous multi-sensory robotic assistant for a drinking task, in: *2019 IEEE 16th International Conference on Rehabilitation Robotics (ICORR)*, IEEE, 2019, pp. 210–216.
- [17] M. Linardakis, I. Varlamis, G. T. Papadopoulos, Survey on hand gesture recognition from visual input, *arXiv preprint arXiv:2501.11992* (2025).
- [18] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, M. Grundmann, Mediapipe hands: On-device real-time hand tracking, *arXiv preprint arXiv:2006.10214* (2020).
- [19] Y. Meng, H. Jiang, N. Duan, H. Wen, Real-time hand gesture monitoring model based on mediapipe’s registerable system, *Sensors* 24 (2024) 6262.
- [20] R. Kumar, A. Bajpai, A. Sinha, Mediapipe and cnns for real-time asl gesture recognition, *arXiv preprint arXiv:2305.05296* (2023).
- [21] B. Subramanian, B. Olimov, S. M. Naik, S. Kim, K.-H. Park, J. Kim, An integrated mediapipe-optimized gru model for indian sign language recognition, *Scientific Reports* 12 (2022) 11964.
- [22] O. Köpüklü, A. Gunduz, N. Kose, G. Rigoll, Real-time hand gesture detection and classification using convolutional neural networks, in: *2019 14th IEEE international conference on automatic face & gesture recognition (FG 2019)*, IEEE, 2019, pp. 1–8.
- [23] J. Singha, K. Das, Hand gesture recognition based on karhunen-loeve transform, *arXiv preprint arXiv:1306.2599* (2013).
- [24] Yaseen, O.-J. Kwon, J. Kim, S. Jamil, J. Lee, F. Ullah, Next-gen dynamic hand gesture recognition: Mediapipe, inception-v3 and lstm-based enhanced deep learning model, *Electronics* 13 (2024) 3233.
- [25] J. Björklund, Hand gesture recognition in real time-fast, accurate hand gesture recognition of static and dynamic hand gestures for on device classification (2024).
- [26] N. Schlüsener, M. Bücken, Fast learning of dynamic hand gesture recognition with few-shot learning models, *arXiv preprint arXiv:2212.08363* (2022).
- [27] MARRtino Project, Marrtino robot project, <https://www.marrtino.org/>, 2025. Accessed: 2025-05-30.
- [28] AWS Robotics, Aws robomaker hospital world, <https://github.com/aws-robotics/aws-robomaker-hospital-world>, 2021. GitHub repository, Accessed: 2025-05-30.
- [29] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science robotics* 7 (2022) eabm6074.
- [30] Navigation 2 Project, Navigation 2 documentation, <https://docs.nav2.org/>, 2025. Accessed: 2025-05-30.

- [31] J. Materzynska, G. Berger, I. Bax, R. Memisevic, The jester dataset: A large-scale video dataset of human gestures, in: *Proceedings of the IEEE/CVF international conference on computer vision workshops*, 2019, pp. 0-0.
- [32] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., Scikit-learn: Machine learning in python, the *Journal of machine Learning research* 12 (2011) 2825-2830.